КУРСОВАЯ РАБОТА

Тема: Создание сайта для опросов

ВВЕДЕНИЕ

Цифровизация образовательной, корпоративной и общественной среды привела к резкому росту спроса на инструменты быстрого сбора обратной связи. Опросы используются в учебных курсах, внутри команд разработки, в HR-процессах, в маркетинговых исследованиях и при принятии управленческих решений. При этом практическая ценность опроса определяется не только возможностью «задать вопрос», но и качеством всего жизненного цикла: от аутентификации пользователя и удобства интерфейса до надежного хранения данных, файловых вложений, статистики и устойчивой обработки ошибок.

В рамках курсовой работы была поставлена задача реализовать полноценную информационную систему опросов в командном формате. Команда состояла из трех участников, что позволило разделить зоны ответственности и построить проект по принципам промышленной разработки: backend, frontend и опциональные расширения. Такой формат важен с учебной точки зрения, поскольку максимально приближен к реальной разработке программных продуктов, где результат зависит не от одного модуля, а от согласованной работы нескольких направлений.

Данная работа содержит две основные главы. Первая глава является общей: в ней описываются цели проекта, единая архитектура, командное распределение задач, интерфейсы взаимодействия между компонентами, а также общие требования к качеству и функциональности. Вторая глава полностью посвящена моей части проекта — клиентскому приложению. В ней подробно рассматриваются реализованные функции, соответствующие критериям оценки: клиентское приложение, авторизация на клиенте, интеграция API, асинхронные запросы, обработка ошибок и работа с файлами.

Актуальность темы определяется несколькими факторами:

? высокой востребованностью web-систем для опросов и аналитики;

? необходимостью безопасной авторизации в публичных и корпоративных приложениях;

? важностью устойчивого пользовательского интерфейса при нестабильной сети и ошибках API;

? практической ценностью навыков интеграции frontend и backend в единую систему.

Цель работы: разработать и продемонстрировать клиент-серверную систему опросов, соответствующую учебным требованиям, с акцентом на полноценную реализацию клиентского приложения и его интеграцию с серверной частью.

Для достижения цели были поставлены задачи:

1. Сформировать общую архитектуру приложения и распределить командные роли.
2. Обеспечить интерфейсное и техническое согласование между frontend и backend.
3. Реализовать клиентскую часть с учетом требований к функциональности и UX.
4. Реализовать защищенное взаимодействие клиента с API (JWT, хранение токена, обновление сессии).
5. Подключить все необходимые методы API для работы с опросами, голосованием и файлами.
6. Реализовать асинхронные запросы с отображением состояний загрузки.
7. Внедрить централизованную обработку ошибок с UI-уведомлениями.
8. Обеспечить работу с файлами: загрузка, отображение, скачивание, валидация.
9. Провести проверку работоспособности и подготовить отчетный материал.

Практическая значимость работы заключается в получении работающего программного решения и формировании устойчивых навыков командной web-разработки: декомпозиция задач, интеграция модулей, работа с API, обработка исключений и итоговая системная документация.

ГЛАВА 1. ОБЩАЯ ЧАСТЬ ПРОЕКТА (ДЛЯ ВСЕЙ КОМАНДЫ)

1.1 Постановка общей задачи проекта

Командный проект был направлен на создание системы онлайн-опросов, в которой пользователь может:

? зарегистрироваться и авторизоваться;

? просматривать список опросов;

? создавать, редактировать и удалять опросы (при наличии прав);

? проходить опросы, отправляя ответы;

? просматривать статистику по результатам;

? работать с файловыми вложениями (изображениями/дополнительными материалами).

Системе предъявлялись требования как к функциональности, так и к архитектуре: соблюдение REST-подхода, использование Entity Framework, Swagger-документация, JWT-авторизация, клиентская интеграция всех методов и корректная обработка ошибок.

1.2 Командное распределение ролей

Для достижения результата проект был декомпозирован на три блока:

1. Backend-часть (коллега 1):

? реализация серверного API;

? моделирование сущностей и доступ к данным через EF;

? аутентификация и авторизация на JWT;

? реализация CRUD-логики;

? серверные эндпоинты для работы с файлами и статистикой;

? документация API в Swagger.

2. Опциональные задания (коллега 2):

? расширения сверх базового задания;

? доработка архитектурных и нефункциональных аспектов;

? дополнительные возможности (в зависимости от выбранных задач).

3. Frontend-часть (моя зона ответственности):

? разработка пользовательского web-клиента;

? маршрутизация и экраны приложения;

? клиентская авторизация и хранение токенов;

? интеграция всех необходимых API методов;

? асинхронные операции и индикация загрузки;

? обработка ошибок и UI-уведомления;

? клиентская работа с файлами.

Такой формат обеспечил параллельную разработку при четком разделении ответственности и понятных точках интеграции.

1.3 Архитектура решения

Приложение построено по классической клиент-серверной модели:

? Серверный уровень: ASP.NET Core Web API.

? Клиентский уровень: Web-приложение на React.

? Хранилище данных: SQLite (через Entity Framework Core).

? Авторизация: JWT access token (и refresh-механизм на клиенте).

? Документация API: Swagger/OpenAPI.

С точки зрения потоков данных архитектура выглядит следующим образом:

1. Пользователь выполняет вход/регистрацию через клиент.

2. Сервер проверяет данные и возвращает токен.

3. Клиент сохраняет токен и добавляет его в Authorization для защищенных запросов.

4. Клиент обращается к API-методам опросов, файлов и статистики.

5. Сервер возвращает данные либо коды ошибок, которые корректно обрабатываются в UI.

Ключевым аспектом архитектуры стало разделение ответственности:

? backend отвечает за бизнес-правила, безопасность и целостность;

? frontend отвечает за UX, валидацию пользовательского ввода, реакцию на состояния сети и понятную визуализацию.

1.4 Технологический стек проекта

Общий стек проекта включал:

? C# / .NET (сервер);

? ASP.NET Core Web API;

? Entity Framework Core;

? SQLite;

? Swagger;

? JWT;

? JavaScript / React / Vite (клиент);

? CSS для интерфейсного оформления;

? SignalR (в части real-time обновлений статистики).

Выбор данного стека обусловлен следующими причинами:

? быстрая разработка и отладка;

? доступная документация;

? хорошая совместимость компонентов;

? достаточная функциональность для учебного проекта и демонстрации реальных практик.

1.5 Общее соответствие критериям задания

Проект как единая система покрывает обязательные требования дисциплины:

? наличие клиентского и серверного приложения;

? авторизация и защищенные методы;

? CRUD для доменных сущностей;

? соблюдение REST-подхода;

? подключенная документация API;

? работа с файлами;

? проверка/тестирование ключевой логики.

Отдельные критерии, относящиеся к frontend, детально раскрыты в главе 2, так как именно они являются моей персональной частью.

1.6 Интеграция модулей и контракты между командами

Чтобы команды работали независимо, но совместимо, были зафиксированы API-контракты:

? единые URL эндпоинтов;

? ожидаемые тела запросов/ответов;

? формат ошибок;

? правила передачи JWT в заголовке Authorization;

? форматы данных для вопросов, вариантов ответа, статистики;

? правила загрузки и выдачи файлов.

В процессе интеграции вносились уточнения, например:

? корректная обработка id опросов на клиенте;

? унификация формата вопросов для передачи и чтения;

? защита от некорректных локальных черновых идентификаторов;

? согласование поведения при 401/403/404.

Эти доработки являются нормальной частью жизненного цикла любого проекта и отражают практическую ценность командной работы.

1.7 Общие требования к качеству

К проекту применялись следующие требования качества:

? функциональная полнота по заданию;

? корректная работа в типовых пользовательских сценариях;

? предсказуемая обработка ошибок;

? устойчивость к частичным сбоям (временные сетевые проблемы, недоступность отдельных методов);

? понятность интерфейса и обратной связи;

? расширяемость кода.

Для соблюдения этих требований использовались:

? модульная структура кода;

? разделение API-слоя и UI-слоя на клиенте;

? централизованная обработка HTTP-ошибок;

? контроль доступности действий по роли/владению ресурсом;

? поэтапная ручная проверка основных сценариев.

1.8 Общие риски и способы их минимизации

В ходе проекта были выявлены типовые риски:

1. Неполное совпадение контрактов frontend/backend.
2. Непредвиденные форматы данных в ответах API.
3. Проблемы авторизации (просроченные или некорректные токены).
4. Сбойные сценарии при работе с файлами.
5. Локальные черновые данные, не предназначенные для серверной валидации.

Для минимизации рисков использовались:

? регулярная синхронизация между участниками;

? добавление fallback-обработки на клиенте;

? явные пользовательские уведомления о причинах ошибок;

? доработка валидации перед отправкой данных;

? очистка и нормализация данных на обоих слоях.

1.9 Выводы по главе 1

Первая глава показала, что проект реализован как единая командная система с четким распределением ответственности и интеграцией результатов. Общая архитектура обеспечивает работоспособность и масштабируемость

решения, а согласование API-контрактов позволило объединить backend, frontend и опциональные модули в единый программный продукт.

Далее во второй главе подробно рассматривается моя персональная часть — клиентское приложение, его функции, техническая реализация и соответствие критериям оценки.

ГЛАВА 2. МОЯ ЧАСТЬ: КЛИЕНТСКОЕ ПРИЛОЖЕНИЕ И ПОЛЬЗОВАТЕЛЬСКАЯ ЛОГИКА

2.1 Цели и границы моей части работы

Моя персональная зона ответственности охватывала полный frontend-блок проекта. Ключевая цель состояла в том, чтобы пользователь мог выполнить весь прикладной сценарий работы с системой через удобный web-интерфейс: от входа в аккаунт до создания/прохождения опроса и работы с вложениями.

Перечень критериев, закрываемых моей частью:

? Реализовано клиентское приложение (WPF/Avalonia/Web/Mobile) — 8 баллов.

? Реализована авторизация на клиенте (хранение и обновление токена) — 5 баллов.

? Интеграция всех API-методов — 7 баллов.

? Асинхронные вызовы API (async/await, обработка загрузки) — 3 балла.

? Обработка ошибок (401, 404, сетевые ошибки с UI-уведомлениями) — 3 балла.

? Реализована работа с файлами (загрузка, отображение, скачивание) — 4 балла.

2.2 Реализация клиентского web-приложения (8 баллов)

Клиентское приложение реализовано как SPA на React. Структура интерфейса организована вокруг маршрутов, соответствующих пользовательским сценариям:

? главная страница;

? страница входа;

? страница регистрации;

? список опросов;

? просмотр деталей опроса;

? создание нового опроса;

? редактирование существующего опроса;

? статистика по опросу;

? профиль пользователя;

? fallback-страница «не найдено».

Ключевые интерфейсные требования:

? доступность основных действий из навигации;

? логическая последовательность экранов;

? минимизация числа лишних шагов;

? единообразие кнопок, уведомлений и сообщений об ошибках.

Особое внимание уделялось не только «красоте» UI, но и его стабильности: при ошибках API пользователь не теряет контекст, а получает ясное сообщение и возможность повторить действие.

[МЕСТО ДЛЯ РИСУНКА 2.1: Главная страница web-клиента]

[МЕСТО ДЛЯ РИСУНКА 2.2: Маршрут «Опросы» со списком карточек]

[МЕСТО ДЛЯ РИСУНКА 2.3: Страница деталей опроса]

2.3 Клиентская авторизация: хранение и обновление токена (5 баллов)

На клиенте реализован полноценный цикл авторизации:

1. Регистрация нового пользователя.

2. Вход по учетным данным.

3. Получение access token от сервера.

4. Сохранение токена в localStorage.

5. Автоматическая подстановка Bearer-токена в защищенные запросы.

6. Обновление access token при получении 401 (через refresh-механизм).

7. Очистка токена при logout.

Технически реализован централизованный HTTP-клиент, который:

? формирует заголовки;

? обрабатывает коды ответа;

? инициирует refresh при необходимости;

? не дублирует сетевую логику по страницам.

Эта архитектура позволяет поддерживать единое поведение авторизации и упрощает сопровождение кода.

Дополнительно реализованы защитные механизмы:

? ограничение доступа к защищенным маршрутам без токена;

? автоматический выход при критических ошибках сессии;

? предотвращение отправки невалидных токенов в API.

[МЕСТО ДЛЯ РИСУНКА 2.4: Экран входа и успешной авторизации]

[МЕСТО ДЛЯ РИСУНКА 2.5: Пример protected route при неавторизованном доступе]

2.4 Интеграция всех API-методов (7 баллов)

Интеграция API выполнена через отдельные клиентские модули (auth, polls, votes, files), что дало следующие преимущества:

? единый стиль сетевых вызовов;

? изоляция бизнес-логики от компонентов UI;

? удобство тестирования и отладки;

? минимизация дублирования кода.

Интегрированные методы включают:

? `POST /auth/register` — регистрация;

? `POST /auth/login` — вход;

? `POST /auth/refresh` — обновление токена;

? `POST /auth/logout` — выход;

? `GET /surveys` — список опросов (с пагинацией/поиском);

? `GET /surveys/{id}` — детали опроса;

? `POST /surveys` — создание;

? `PUT /surveys/{id}` — редактирование;

? `DELETE /surveys/{id}` — удаление;

? `POST /surveys/{id}/upload-image` — загрузка файла;

? `GET /surveys/{id}/image` — получение файла;

? `POST /votes/{surveyId}` — отправка ответов;

? `GET /votes/{surveyId}/stats` — получение статистики.

Интеграция покрывает весь пользовательский цикл:

авторизация → выбор опроса → заполнение → отправка → просмотр статистики.

[МЕСТО ДЛЯ РИСУНКА 2.6: Создание опроса через форму]

[МЕСТО ДЛЯ РИСУНКА 2.7: Редактирование существующего опроса]

[МЕСТО ДЛЯ РИСУНКА 2.8: Страница статистики]

2.5 Асинхронные вызовы и состояния загрузки (3 балла)

Все API-запросы выполняются асинхронно (async/await). Для каждого сценария реализованы состояния:

? `loading` — когда данные загружаются;

? `busy` — когда выполняется операция изменения данных;

? `error` — когда запрос завершился ошибкой.

Практическое значение:

? пользователь всегда понимает, что происходит;

? уменьшается вероятность повторных нажатий и дублей запросов;

? интерфейс остается отзывчивым даже при медленной сети.

Примеры реализации:

? на страницах списка и деталей опроса отображается «Загрузка...»;

? кнопки сохранения/отправки блокируются в момент запроса;

? после успешной операции интерфейс обновляется без ручной перезагрузки.

[МЕСТО ДЛЯ РИСУНКА 2.9: Индикация загрузки в форме опроса]

2.6 Обработка ошибок и пользовательские уведомления (3 балла)

Обработка ошибок построена на сочетании централизованного и контекстного подходов:

1. Централизованно в API-клиенте:

? анализ HTTP-кода;

? нормализация текстов ошибок;

? запуск refresh при 401;

? проброс исключения в UI-компонент.

2. Локально в компонентах:

? показ toast-уведомлений;

? вывод ошибки рядом с формой;

? переключение сценария в безопасный режим (например, fallback).

Обрабатываемые случаи:

? `401 Unauthorized` — попытка обновить токен, затем повтор запроса;

? `403 Forbidden` — запрет операции (например, редактирование чужого опроса);

? `404 Not Found` — ресурс не найден;

? `400 Bad Request` — ошибки валидации;

? сетевые ошибки (`TypeError`/потеря соединения).

Важный практический эффект: приложение не «падает» из-за ошибки одного запроса, пользователь сохраняет контроль над процессом.

[МЕСТО ДЛЯ РИСУНКА 2.10: Пример toast-уведомления об ошибке API]

2.7 Работа с файлами: загрузка, отображение, скачивание (4 балла)

Файловая подсистема клиента включает:

? выбор изображения для опроса;

? проверку расширения и размера;

? отправку файла через FormData;

? отображение изображения в карточке/деталях;

? скачивание файла пользователем.

Реализованы проверки корректности:

? запрещена отправка неподдерживаемых форматов;

? ограничен размер файла;

? запрещены попытки загрузки файла к «локальному черновику» без серверного id;

? обеспечено корректное формирование URL для предпросмотра.

Данная часть особенно важна для практических систем, где файлы часто являются неотъемлемой частью контента.

[МЕСТО ДЛЯ РИСУНКА 2.11: Загрузка изображения в редакторе опроса]

[МЕСТО ДЛЯ РИСУНКА 2.12: Отображение прикрепленного изображения]

[МЕСТО ДЛЯ РИСУНКА 2.13: Кнопка скачивания файла]

2.8 Реализация пользовательского сценария «Опросы»

Сценарий пользователя реализован как последовательность:

1. Пользователь авторизуется.
2. Переходит в список опросов.
3. Открывает детали конкретного опроса.
4. Заполняет ответы по вопросам (варианты или свободный текст).
5. Отправляет результаты.
6. Переходит к статистике.

Особенности реализации:

? поддерживаются разные типы вопросов;

? есть валидация заполненности перед отправкой;

? реализовано ограничение доступа к действиям владельца;

? доступ к статистике управляется на уровне UI и API-логики.

2.9 Реализация статистики и обновление данных

Статистика опроса реализована в отдельном интерфейсе и включает:

? общее количество ответов;

? распределение по вариантам;

? процентные значения;

? отображение свободных ответов.

Обновление статистики выполняется по API, а также поддерживается механизм обновления по событию. Это позволяет пользователю видеть актуальные данные без ручного обновления страницы.

Визуально статистика представлена в удобной форме (карточки, полосы процентов, списки ответов), что повышает читаемость результатов.

[МЕСТО ДЛЯ РИСУНКА 2.14: Страница статистики с распределением ответов]

2.10 Внутренняя структура frontend-кода

Клиентское приложение организовано по слоям:

? `pages/` — экранные компоненты;

? `components/` — переиспользуемые UI-блоки;

? `api/` — работа с backend;

? `lib/` — вспомогательные функции (токены, обработка ошибок, локальные хранилища);

? `contexts/` — глобальные состояния/уведомления.

Преимущества такой структуры:

? читаемость;

? быстрое добавление новых функций;

? локализация изменений без каскадной правки всего проекта;

? удобство командной поддержки.

2.11 Технические проблемы в процессе разработки и их решение

В процессе интеграции были обнаружены и устранены реальные проблемы.

Проблема 1: Невалидные локальные id (`local_poll_*`) в запросах к backend

Симптом: backend возвращал `400` с ошибкой валидации id.

Причина: локальный черновик опроса не должен отправляться на сервер как integer-id.

Решение:

? добавлена проверка серверного id на клиенте;

? локальные черновики обрабатываются локально;

? исключены запросы `PUT/GET/DELETE` на backend для несерверных id.

Проблема 2: JWT malformed в логах API

Симптом: в консоли сервера появлялась ошибка `IDX14100`.

Причина: в заголовок Authorization попадал невалидный токен (например, остаток локальной сессии).

Решение:

? очистка некорректных токенов;

? повторная авторизация;

? контроль формата токена перед отправкой.

Проблема 3: Мерцание страницы опроса

Симптом: страница циклически перезагружалась.

Причина: неверные зависимости `useEffect`, вызывавшие повторные запросы.

Решение:

? стабилизация зависимостей эффекта;

? устранение лишних триггеров перезагрузки;

? повторная проверка сценария открытия деталей опроса.

Данные исправления повысили стабильность интерфейса и качество пользовательского опыта.

2.12 Тестирование клиентской части

Проверка выполнялась по сценарному подходу (ручное функциональное тестирование):

Сценарий А. Авторизация

? Открыть страницу входа.

? Ввести корректные данные.

? Проверить переход к защищенным разделам.

? Обновить страницу и убедиться, что сессия сохранена.

Ожидаемый результат: вход успешен, токен сохранен, защищенные маршруты доступны.

Сценарий B. Создание опроса

? Открыть «Создать опрос».

? Заполнить заголовок, описание, вопросы.

? Сохранить.

Ожидаемый результат: опрос появляется в списке.

Сценарий C. Редактирование/удаление

? Открыть свой опрос.

? Изменить содержание, сохранить.

? Проверить отображение изменений.

? Удалить опрос.

Ожидаемый результат: операции доступны только владельцу, данные корректно изменяются.

Сценарий D. Прохождение опроса

? Открыть опрос.

? Заполнить ответы.

? Отправить.

Ожидаемый результат: ответ принят, доступна статистика.

Сценарий E. Работа с файлами

? Загрузить допустимый файл.

? Проверить отображение.

? Скачать файл.

Ожидаемый результат: файл успешно загружен, отображается и скачивается.

Сценарий F. Ошибки API

? Смоделировать 401/404/сетевую ошибку.

? Проверить, что UI показывает понятное сообщение.

Ожидаемый результат: приложение не падает, пользователь получает корректное уведомление.

[МЕСТО ДЛЯ РИСУНКА 2.15: Пример тестового сценария/результата в интерфейсе]

2.13 Достижение критериев оценки по моей части

Ниже приведено соответствие выполненных работ критериям:

1. Реализовано клиентское приложение — выполнено.
2. Реализована авторизация на клиенте (хранение/обновление токена) — выполнено.
3. Интеграция всех API-методов — выполнено.
4. Асинхронные вызовы API и обработка загрузки — выполнено.
5. Обработка ошибок (401, 404, сетевые) с UI-уведомлениями — выполнено.
6. Работа с файлами (загрузка, отображение, скачивание) — выполнено.

Таким образом, моя часть закрывает полный набор обязательных требований frontend-блока.

2.14 Перспективы развития клиентской части

Возможные улучшения:

? расширение аналитики (фильтры, экспорт отчетов);

? более детальная ролевая модель доступа;

? улучшенная доступность интерфейса (a11y);

? добавление e2e-тестов на ключевые пользовательские сценарии;

? офлайн-режим с последующей синхронизацией;

? мультиязычность интерфейса.

Эти направления могут быть использованы как продолжение проекта или основа для ВКР/дальнейшей практической разработки.

2.15 Выводы по главе 2

Во второй главе показано, что клиентская часть реализована как полноценный рабочий модуль, интегрированный с серверной системой. В ходе разработки были реализованы все обязательные функции, устранены проблемные сценарии и обеспечена устойчивость интерфейса в реальных условиях эксплуатации.

Моя часть проекта не ограничилась «визуальной оболочкой»: она включает полноценную прикладную логику, сетевое взаимодействие, авторизацию, файловые сценарии и обработку ошибок, что подтверждает завершенность frontend-решения и соответствие требованиям задания.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была создана рабочая клиент-серверная система для проведения онлайн-опросов. Проект выполнен в

командном формате с четким распределением ролей и согласованной интеграцией результатов.

Первая глава представила общую архитектуру и командную организацию разработки. Показано, что проект имеет завершенную структуру, логичную декомпозицию задач и единые технические контракты между частями системы.

Во второй главе подробно раскрыта моя персональная работа по frontend-направлению. Реализованы все критерии клиентской части:

? полноценное web-приложение;

? авторизация с хранением и обновлением токенов;

? интеграция API-методов;

? асинхронные вызовы и состояния загрузки;

? обработка ошибок с UI-уведомлениями;

? работа с файлами.

Дополнительно в процессе разработки были выявлены и устранены практические проблемы интеграции (невалидные локальные id, ошибки JWT, мерцание страницы), что повысило надежность приложения.

Итог работы: поставленная цель достигнута, все обязательные задачи моей части выполнены, клиентское приложение готово к демонстрации и соответствует учебным требованиям.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ (ШАБЛОН)

1. Документация ASP.NET Core Web API.
2. Документация Entity Framework Core.
3. Документация Swagger / OpenAPI.
4. Документация React.
5. Документация Vite.
6. Документация по JWT (RFC 7519).
7. Методические указания по выполнению курсовой работы.

ПРИЛОЖЕНИЯ

Приложение А. Скриншоты клиентского приложения (вставить)

? Рисунок 2.1 — Главная страница.

? Рисунок 2.2 — Список опросов.

? Рисунок 2.3 — Детали опроса.

? Рисунок 2.4 — Авторизация.

? Рисунок 2.5 — Защищенный маршрут.

? Рисунок 2.6 — Создание опроса.

? Рисунок 2.7 — Редактирование опроса.

? Рисунок 2.8 — Статистика.

? Рисунок 2.9 — Состояние загрузки.

? Рисунок 2.10 — Сообщение об ошибке.

? Рисунок 2.11 — Загрузка файла.

? Рисунок 2.12 — Отображение файла.

? Рисунок 2.13 — Скачивание файла.

? Рисунок 2.14 — Итоговая статистика.

? Рисунок 2.15 — Тестовый сценарий.

Приложение Б. Рекомендации по доведению до 20–25 страниц в DOCX

Чтобы итоговый документ стабильно занял 20–25 страниц, рекомендуется:

1. Использовать формат:

? шрифт Times New Roman, 14 pt;

? межстрочный интервал 1.5;

? выравнивание по ширине;

? абзацный отступ 1.25 см;

? поля по требованиям методички.

2. Вставить 12–15 скриншотов в главу 2 (каждый с подписью «Рисунок 2.x ...»).

3. Оставить автоматическое оглавление и обновить нумерацию страниц.

4. При необходимости расширить:

? подраздел 2.12 «Тестирование» подробными таблицами тест-кейсов;

? подраздел 1.6 описанием интеграционных диаграмм;

? приложение с листингами ключевых фрагментов кода.

При указанном оформлении и добавлении скриншотов документ обычно получается в диапазоне 20–25 страниц.

Приложение В. Подробные тест-кейсы клиентской части

Ниже приведен расширенный набор тест-кейсов для демонстрации полноты проверки frontend-функционала. При вставке в DOCX этот раздел также увеличивает итоговый объем отчета и подтверждает практическую проработку сценариев.

B.1 Авторизация и управление сессией

TC-AUTH-01. Успешный вход

? Предусловие: пользователь зарегистрирован.

? Шаги: открыть страницу входа, ввести корректные email/пароль, нажать «Войти».

? Ожидаемый результат: пользователь попадает на защищенный раздел, токен сохранен, кнопка «Выйти» отображается.

TC-AUTH-02. Ошибка входа при неверном пароле

? Предусловие: пользователь зарегистрирован.

? Шаги: ввести корректный email и неверный пароль.

? Ожидаемый результат: отображается сообщение об ошибке авторизации, доступ не предоставлен.

TC-AUTH-03. Доступ к защищенному маршруту без токена

? Предусловие: localStorage очищен.

? Шаги: открыть URL защищенной страницы напрямую.

? Ожидаемый результат: редирект на страницу входа.

TC-AUTH-04. Логаут

? Предусловие: пользователь авторизован.

? Шаги: нажать «Выйти».

? Ожидаемый результат: токены удалены, пользователь возвращается в публичный раздел.

TC-AUTH-05. Поведение при 401 и refresh

? Предусловие: access token истек, refresh token валиден.

? Шаги: вызвать защищенный метод.

? Ожидаемый результат: выполняется refresh, исходный запрос успешно повторяется.

TC-AUTH-06. Невалидный refresh token

? Предусловие: refresh token испорчен.

? Шаги: вызвать защищенный метод.

? Ожидаемый результат: refresh неудачен, сессия очищается, пользователь перенаправляется на вход.

B.2 Работа со списком и деталями опросов

TC-POLL-01. Загрузка списка опросов

? Шаги: открыть страницу «Опросы».

? Ожидаемый результат: отображается список, пагинация и кнопки действий в корректном состоянии.

TC-POLL-02. Поиск по названию

? Шаги: ввести поисковый запрос, выполнить поиск.

? Ожидаемый результат: отображаются только опросы, удовлетворяющие фильтру.

TC-POLL-03. Переход в детали опроса

? Шаги: выбрать карточку опроса.

? Ожидаемый результат: открывается страница деталей, отображаются вопросы и описание.

TC-POLL-04. Отсутствующий id опроса

? Шаги: открыть URL с несуществующим id.

? Ожидаемый результат: пользователь видит сообщение «Опрос не найден».

TC-POLL-05. Локальный id черновика

? Предусловие: есть локальный черновик `local_poll_*`.

? Шаги: открыть его страницу, выполнить просмотр/редактирование.

? Ожидаемый результат: операции выполняются локально, backend-запрос с невалидным id не отправляется.

B.3 Создание и редактирование опроса

TC-EDIT-01. Создание опроса с валидными данными

? Шаги: заполнить заголовок, описание, минимум один вопрос, сохранить.

? Ожидаемый результат: опрос создается, появляется в списке.

TC-EDIT-02. Пустой заголовок

? Шаги: оставить заголовок пустым, нажать «Сохранить».

? Ожидаемый результат: локальная валидация блокирует отправку, отображается предупреждение.

TC-EDIT-03. Вопрос без текста

? Шаги: оставить один из вопросов пустым.

? Ожидаемый результат: сохранение недоступно до исправления.

TC-EDIT-04. Вопрос с вариантами: меньше минимального числа ответов

? Шаги: оставить только один вариант в choice-вопросе.

? Ожидаемый результат: отображается предупреждение о минимальном числе вариантов.

TC-EDIT-05. Редактирование существующего опроса владельцем

? Шаги: открыть свой опрос, изменить поля, сохранить.

? Ожидаемый результат: изменения сохранены и отображаются после перезагрузки.

TC-EDIT-06. Редактирование чужого опроса

? Предусловие: пользователь не является владельцем.

? Шаги: попытка открыть `/edit` вручную.

? Ожидаемый результат: действие блокируется, отображается ошибка прав доступа.

TC-EDIT-07. Удаление своего опроса

? Шаги: нажать «Удалить», подтвердить действие.

? Ожидаемый результат: опрос удаляется из списка.

B.4 Прохождение опроса и статистика

TC-VOTE-01. Прохождение опроса со всеми заполненными ответами

? Шаги: заполнить каждый вопрос, отправить.

? Ожидаемый результат: backend принимает данные, пользователь видит успешное уведомление.

TC-VOTE-02. Незаполненный обязательный ответ

? Шаги: оставить вопрос без ответа и отправить форму.

? Ожидаемый результат: отправка блокируется, отображается предупреждение.

TC-VOTE-03. Свой вариант для choice-вопроса

? Шаги: выбрать «Свой вариант», ввести текст, отправить.

? Ожидаемый результат: ответ учитывается в статистике как custom answer.

TC-VOTE-04. Повторная отправка ответов тем же пользователем

? Шаги: отправить повторные ответы на тот же опрос.

? Ожидаемый результат: запись обновляется по responder key, дубль не создается.

TC-STATS-01. Просмотр статистики после голосования

? Шаги: перейти в раздел статистики.

? Ожидаемый результат: отображаются суммарные ответы, проценты и списки текстовых ответов.

TC-STATS-02. Доступ к статистике до прохождения опроса

? Предусловие: пользователь не владелец и не проходил опрос.

? Шаги: открыть страницу статистики.

? Ожидаемый результат: показывается уведомление об ограничении доступа.

B.5 Работа с файлами

TC-FILE-01. Загрузка валидного изображения

? Шаги: выбрать JPG/PNG корректного размера, сохранить опрос.

? Ожидаемый результат: файл загружен, превью отображается.

TC-FILE-02. Невалидный формат файла

? Шаги: выбрать неподдерживаемое расширение.

? Ожидаемый результат: загрузка блокируется, показывается сообщение об ошибке.

TC-FILE-03. Превышение лимита размера

? Шаги: выбрать файл больше допустимого.

? Ожидаемый результат: загрузка не выполняется, выводится предупреждение.

TC-FILE-04. Скачивание файла

? Шаги: нажать «Скачать файл» в карточке/деталях опроса.

? Ожидаемый результат: файл корректно скачивается на устройство.

TC-FILE-05. Попытка загрузки к локальному черновику

? Предусловие: id опроса локальный (`local_poll_*`).

? Шаги: выбрать файл и попытаться загрузить.

? Ожидаемый результат: клиент блокирует запрос к backend и выводит понятное сообщение.

B.6 Обработка ошибок и UX-устойчивость

TC-ERR-01. Ошибка 404 от API

? Шаги: обратиться к отсутствующему ресурсу.

? Ожидаемый результат: UI показывает понятное сообщение «Ресурс не найден».

TC-ERR-02. Ошибка 500 от API

? Шаги: смоделировать внутреннюю ошибку сервера.

? Ожидаемый результат: пользователь видит уведомление о серверной ошибке и может повторить действие.

TC-ERR-03. Потеря сети

? Шаги: отключить сеть и выполнить запрос.

? Ожидаемый результат: показывается уведомление о сетевой ошибке, интерфейс не ломается.

TC-ERR-04. Многоразовое повторение ошибки

? Шаги: несколько раз повторить неуспешный запрос.

? Ожидаемый результат: приложение остается стабильным, уведомления не приводят к зависанию UI.

TC-ERR-05. Проверка на мерцание страницы опроса

? Шаги: открыть детали опроса и наблюдать поведение страницы.

? Ожидаемый результат: нет бесконечных циклов загрузки, запрос выполняется однократно по сценарию.

Приложение Г. Матрица «требование → реализация → подтверждение»

| № | Требование | Реализация в клиенте | Подтверждение |

|---|---|---|---|

| 1 | Клиентское приложение | React SPA с маршрутизацией и экранами | Демонстрация экранов + скриншоты |

| 2 | Клиентская авторизация | Login/Register, хранение токенов, logout | Сценарии TC-AUTH-* |

| 3 | Обновление токена | Refresh flow при 401 | TC-AUTH-05, TC-AUTH-06 |

| 4 | Интеграция API | auth/polls/votes/files модули | Сценарии TC-POLL-*, TC-VOTE-* |

| 5 | Async/await | Асинхронные запросы, loading/busy | TC-POLL-01, TC-EDIT-01 |

| 6 | Обработка ошибок | Централизованный ApiError + toast | TC-ERR-* |

| 7 | Работа с файлами | Upload/preview/download + валидация | TC-FILE-* |

| 8 | Доступ по ролям/владению | Ограничение edit/delete чужих опросов | TC-EDIT-06 |

| 9 | Статистика | Отдельный экран статистики, агрегаты | TC-STATS-* |

|10 | Устойчивость UX | Отсутствие мерцания, понятные сообщения | TC-ERR-05 + ручная проверка |

Приложение Д. Рекомендованный набор изображений для 2 главы

Чтобы визуально усилить отчет и довести объем до требуемых 20–25 страниц, рекомендуется вставить не менее 15–18 скриншотов:

1. Главная страница.

2. Страница регистрации.

3. Страница входа.

4. Список опросов.

5. Карточка опроса без обложки.

6. Карточка опроса с обложкой.

7. Детали опроса.

8. Форма прохождения опроса (варианты ответа).

9. Форма прохождения опроса (свободный ответ).

10. Страница статистики (общие карточки).

11. Страница статистики (распределение по вариантам).

12. Редактор опроса: создание.

13. Редактор опроса: редактирование.

14. Загрузка изображения.

15. Пример успешного уведомления.

16. Пример ошибки 404/401.

17. Пример ограничения доступа для не-владельца.

18. Пример скачивания файла.

Каждое изображение оформлять подписью вида:

«Рисунок 2.x — Название экрана/сценария».

Приложение Е. Как формально выйти на 20–25 страниц

Если требуется строго обеспечить диапазон 20–25 страниц по методическим требованиям, рекомендуется следующий комплект:

? Основной текст (главы + введение + заключение): 14–17 страниц.

? Скриншоты в главе 2 с подписями: 5–7 страниц.

? Приложения (тест-кейсы и матрица): 2–3 страницы.

В сумме получается целевой диапазон 20–25 страниц без искусственного «раздувания» и с сохранением содержательной ценности отчета.